# 3D_vision

## 1D

1. Projective geometry on a line, $\mathrm{P}^2$.
2. In a 1D world, a point in homogeneous coordinates can be represented as $x' = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$, **1 degrees of freedom**.
3. if $x_2 = 0$, the point will be a point at infinity, or an ideal point.
4. A projective transformation of a line is represented by $H_{2\times2}$, i.e.
    1. $x' = H_{2\times2}x$
    2. $H_{2\times2}$ has **three degrees of freedom**, corresponding to four elements less one for scaling and can be computed from three points.
5. Cross ratio:
    1. $Cross(x_1, x_2, x_3, x_4) = \frac{|x_1x_2||x_3x_4|}{|x_1x_3||x_2x_4|}$, where $x_ix_j = \det \begin{bmatrix} x_{i1} & x_{j1} \\ x_{i2} & x_{j2} \end{bmatrix}$
    2. $x' = H_{2\times2}x$ the cross ratio must be preserved, i.e. should be the same number.
    3. $Cross(x'_1, x'_2, x'_3, x'_4) = cross(x_1, x_2, x_3, x_4)$.
    4. Proof: substitute $x' = H_{2\times2}x$, into the formula $Cross(x'_1, x'_2, x'_3, x'_4)$.

## 2D

1. Homogenize a point in 2D $\begin{pmatrix} x \\ y \end{pmatrix}$ by adding a new dimension, $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$ and replacing $x$ with $\frac{x_1}{x_3}$ and $y$ with $\frac{x_2}{x_3}$. Or equivalently add a new dimension and now can multiply all dimension by a scale $k$, $\begin{pmatrix} kx \\ ky \\ k \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ w' \end{pmatrix}$.
2. Points at infinity:
    1. Homogeneous coordinates of the form $[x, y, 0]^T$ do not correspond to a point in the Cartesian plane.
    2. They correspond to the unique point at infinity in the direction $[x, y]^T$.
    3. Consider the homogenous coordinates $[a + tx, b + ty, 1]$ or equivalently $[\frac{a}{t} + x, \frac{b}{t} + y, \frac{1}{t}]^T$.
    4. As $t$ tends to infinity, homogeneous coordinates become $[x, y, 0]^T$.
3. From the equation of a line in 2D,
4. $$\begin{aligned} ax + by + c &= 0 \\ a(kx) + b(ky) + c(k) &= 0 \\ ax_1 + bx_2 + cx_3 &= 0 \\ l_1x_1 + l_2x_2 + l_3x_3 &= 0 \end{aligned}$$
5. In $\mathrm{P}^2$ both a line $\begin{pmatrix} l_1 \\ l_2 \\ l_3 \end{pmatrix}$ and a point $\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$ has **two degrees of freedom**.
6. Intersection of a point and a line or a line and a point in homogeneous coordinates,

$$l^Tx = 0$$

7. **line formed by two points**
    1. algebraic derivation

$$
\begin{aligned}
l_1x_1 + l_2x_2 + l_3x_3 &= 0 \\
l_1x'_1 + l_2x'_2 + l_3x'_3 &= 0 \\
\frac{l_1}{l_3}\frac{x_1}{x_3} + \frac{l_2}{l_3}\frac{x_2}{x_3} + 1 &= 0 \\
\frac{l_1}{l_3}\frac{x'_1}{x'_3} + \frac{l_2}{l_3}\frac{x'_2}{x'_3} + 1 &= 0 \\
ua + vb &= -1 \\
uc + vd &= -1 \\
\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} &= \begin{bmatrix} -1 \\ -1 \end{bmatrix} \\
\begin{bmatrix} u \\ v \end{bmatrix} &= \frac{\begin{bmatrix} d & -b \\ -c & a \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix}}{ad - bc} \\
u = \frac{l_1}{l_3} &= \frac{x_2x'_3 - x'_2x_3}{x_1x'_2 - x_2x'_1} \\
v = \frac{l_2}{l_3} &= \frac{x'_1x_3 - x_1x'_3}{x_1x'_2 - x_2x'_1} \\
l &= x \times x'
\end{aligned}
$$

2. line formed by two points is the normal vector of the plane formed by the two lines, by taking the cross product (geometric interpretation).

8. **point intersected by two lines**
    1. algebraic derivation,

$$
\begin{aligned}
l_1x_1 + l_2x_2 + l_3x_3 &= 0 \\
l'_1x_1 + l'_2x_2 + l'_3x_3 &= 0
\end{aligned}
$$

    - Symmetrical to the derivation of line formed by two points case.
    - $x = l \times l'$

    2. point intersected by two lines is the direction vector of a line formed by the intersection of two planes (geometric interpretation).
9. Conics
    1. $$
    \begin{aligned}
    ax^2 + bxy + cy^2 + dx + ey + f &= 0 \\
    ax_1^2 + bx_1x_2 + cx_2^2 + dx_1 + ex_2 + f &= 0
    \end{aligned}
    $$
    2. Representation of a conic, 5 dof

$$
\begin{pmatrix} a \\ b \\ c \\ d \\ e \\ f \end{pmatrix}
$$

    3. To represent the relationship between conic and points, we write

$$
x^TCx = 0
$$

    4. $C = \begin{bmatrix} a & b/2 & d/2 \\ b/2 & c & e/2 \\ d/2 & e/2 & f \end{bmatrix}$
    5. The intrinsic camera parameters matrix is an example of a conic (has 5 dof).
    6. C is a homogeneous representation of a conic.
    7. Only ratios of the matrix elements are important, multiplying $C$ by a non-zero scalar has no effect.
    8. The conic has **five degrees of freedom** which can be thought of the ratios $a : b : c : d : e : f$.
    9. Each point $x_i = (x_i, y_i)$ places one constraint on the conic coefficients:

$$
ax_i^2 + bx_iy_i + cy_i^2 + dx_i + ey_i + f = 0
$$

    10. Where

$$
c = (a, b, c, d, e, f)^T
$$

    is the conic $C$ represented by a 6-vector.

$$\begin{bmatrix} x_1^2 & x_1y_1 & y_1^2 & x_1 & y_1 & 1 \\ & & \cdots & & & \\ & & \cdots & & & \\ & & \cdots & & & \\ & & \cdots & & & \end{bmatrix} \begin{pmatrix} a \\ b \\ c \\ d \\ e \\ f \end{pmatrix} = 0$$

11. The $A_{5\times 6}$. The conic is a null vector of this $5 \times 6$ matrix. Can be solved using **SVD**, and taking the eigenvector of $V$ corresponding to the smallest eigenvalue, i.e. the one with the least principal component.
12. This shows that a conic is determined uniquely (up to scale) by five points in general.
13. 

```
|          | Dual       |
| -------- | -------    |
|point     | line       |
|conic     | dual conic |
```

14. Tangent line to conics:
    1. Intersection of point $x$ and tangent line: $l^Tx = 0$
    2. Intersection of point and conic: $x^TCx = 0$
    3. get $l^Tx = x^T(Cx) = 0$, recall $x^Tl = l^Tx$.
    4. Therefore $l = Cx$.
15. Dual conic, based on the definition of a point conic, $x^TCx = 0$, a line conic would have the form: $l^TC^*l = 0$. Where $l$ is a tangent to the conic $C$.
16. 

```
| Conic      | Dual Conic   |
| --------   | -------      |
| $x^TCx=0$  | $l^TC^*l=0$  |
```

17. A dual conic has **five degrees of freedom**, and can be computed from five lines.
18. $l^Tx = 0$,

$$\begin{aligned} x^TCx &= 0 \\ \text{since } l &= Cx, \text{this implies} \\ x &= C^{-1}l \\ (C^{-1}l)^TC(C^{-1}l) &= 0 \\ l^TC^{-T}CC^{-1}l &= 0 \\ l^TC^{-T}l &= 0 \end{aligned}$$

- Recall

$$(A^T)^{-1} = (A^{-1})^T$$

if $A$ is an $n \times n$ invertible matrix. Since $C$ is symmetric $C^{-1} = C^{-T}$.

10. Degenerate conic
    1. first degenerate conic: A degenerate conic made of 2 straight lines.
        1. $C = lm^T + ml^T$
        2. $l$ and $m$ are lines. Points on $l$ satisfy $l^Tx = 0$, similarly $m^Tx = 0$. The two lines themselves is the conic.
        3. Proof: $x^TCx = x^T(lm^T + ml^T)x = (x^Tl)(m^Tx) + (x^Tm)(l^Tx) = 0$
        4. which consists of $x^Tl = l^Tx = l_1x_1 + l_2x_2 + l_3$.
        5. and $m^Tx = x^Tm = m_1x_1 + m_2x_2 + m_3x_3$.
        6. has rank(C) = 2.
    2. Second degenerate conic: made of one line or two duplicate lines.
        1. $C = ll^T + ll^T$
        2. Similarly we have $l^Tx = 0, x^Tx = 0$
        3. $x^TCx = x^T(ll^T + ll^T)x = (x^Tl)(l^Tx) + (x^Tl)(l^Tx) = 0$
        4. which consists of $x^Tl = l^Tx = l_1x_1 + l_2x_2 + l_3$
        5. has rank(C) = 1.
    3. Third degenerate conic: made of two points.
        1. $C^* = xy^T + yx^T$ has rank(C) = 2.

4. Fourth degenerate conic: made of a point or two duplicate points.
    1. $C^* = xx^T + xx^T$ has rank(C)=1.

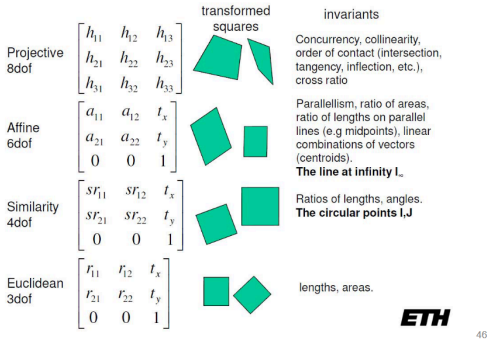
```
| Degenerate Conic      | Degenerate Dual Conic|
| --------------------- | -------------------- |
| $ml^T + lm^T$         | $xy^T + yx^T$        |
| $ll^T + ll^T$         | $xx^T + xx^T$        |
```

11. Planar projective transformations for points
    1. Projectivity, mapping from $\mathrm{P}^2$ to $\mathrm{P}^2$.
    2. such that three points $x_1$, $x_2$, and $x_3$. Why three? (because a plane in 3D has three degrees of freedom?) lie on the same line if and only if $h(x_1)$, $h(x_2)$, $h(x_3)$ do.
    3. $x' = hx \Rightarrow x = h^{-1}x'$.
    4. Define planar projective transformation, a transformation matrix for objects in 2D plane for example points or lines (in homogeneous coordinates, knowing points or lines have two degrees of freedom, and bumping the dimension by one, i.e. from 2 to 3 coordinates, and keeping the same number of dof), we have:
    5. $$\begin{pmatrix} x'_1 \\ x'_2 \\ x'_3 \end{pmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$
    6. or $x' = Hx$. $H$ is a $3 \times 3$ matrix, and is homogeneous (only the ratio of the matrix elements is significant).
    7. $H$ has **eight degrees of freedom**, nine minus one for scaling.
12. Projective transformation of a line
    1. $l^T x = 0$, $l'^T x' = 0$
    2. $l^T (H^{-1}H)x = 0$
    3. $(H^{-T}l)^T x' = 0 \Rightarrow l' = H^{-T}l$
    4. Under a projective transformation $H$, the points that lie on a line $l$, will be transformed to points that lie on the line $l' = H^{-T}l$
    5. This means under the point transformation $x' = Hx$, a line transforms as $\boxed{l' = H^{-T}l}$ or $\boxed{l'^T = l^T H^{-1}}$.
13. Projective transformation of a conic
    1. projective transformation of a point, $x' = Hx \Rightarrow x = H^{-1}x'$
    2. $x^T C x = (H^{-1}x')^T C (H^{-1}x') = x'^T (H^{-T} C H^{-1}) x'$
    3. under a point transformation $x' = Hx$, a conic transforms according to $\boxed{C' = H^{-T}CH^{-1}}$
14. Projective transformation of a dual conic
    1. $l' = H^{-T}l \Rightarrow l = H^T l'$
    2. $l^T C^* l = (H^T l')^T C^* (H^T l') = l'^T (H C^* H^T) l'$
    3. a dual conic $C^*$ transforms to $\boxed{C^{*'} = HC^*H^T}$
15. Hierarchy of transformations:
    1. General transformation for objects in a 2D plane such as points and lines $H_{3\times3}$, with **eight degrees of freedom**
    2. Isometries: transform that preserves Euclidean distance, rotate and translate figures.
        1. $x' = H_E x = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} x = \begin{bmatrix} \epsilon c\theta & -s\theta & t_x \\ \epsilon s\theta & c\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$, where $\epsilon = \pm 1$.
        2. $\epsilon = +1$, orientation preserving.
        3. $\epsilon = -1$, orientation reversing.
        4. Invariants: length, angle, area.
        5. $H_E$ has **three degrees of freedom**.
        6. *preserving lengths.*
    3. Isotropic scaling, similarities additionally (isotropic) scale figures:
        1. $H_S$ has **four degrees of freedom**.
        2. can be computer from two point correspondences.
        3. *preserving ratio of lengths, angles.*
    4. Affinity transform, additionally non-isotropic scale figures:
        1. $H_A$ has **six degrees of freedom**.
        2. can be computed from three point correspondences

3. Note: each point correspondence would give **two degrees of freedom**, hence why the three points will be sufficient to compute this **six degrees of freedom** matrix.
4. can always be decomposed as:
    1. $A = R(\theta)R(-\phi)DR(\phi)$
    2. This decomposition follows directly from singular value decomposition.
5. *Preserving ratio of lengths on parallel lines, parallel lines*

5. Projective transformation: a non-singular linear transformation of homogeneous coordinates, additionally move the line at infinity.
    1. $x' = H_P x = \begin{bmatrix} A & t \\ \vec{v}^T & u \end{bmatrix} x$ where vector $\vec{v} = (v_1, v_2)^T$ and $u$ can be 0.
    2. Note: it is not always possible to make $[v^T \quad u]$ to be $[0 \quad 0 \quad 1]$.
    3. $H_P$ has nine elements with only the ratio of its elements is significant, so the transformation has **eight degrees of freedom**.
    4. can be computed from four point correspondences, with no three collinear on either plane.
    5. It is not possible to distinguish between orientation preserving and orientation reversing projectivities in $\mathrm{P}^2$.
    6. Decomposition of a projective transformation into a pure projective, a pure affine transform and a similarity:
        1. $H = H_S H_A H_P = \begin{bmatrix} sR & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} K & 0 \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} I & 0 \\ v^T & u \end{bmatrix} = \begin{bmatrix} A & t \\ v^T & u \end{bmatrix}$
        2. $A$ is a non-singular matrix given by: $A = sRK + tv^T$
            1. where $s$ is the scale
            2. $R$ is the rotation matrix
            3. $K$ is an upper triangular matrix
            4. $t$ is translation vector
            5. $v^T$ is a $2 \times 1$ vector
        3. Decomposition is valid if $u \neq 0$, and is unique if $s$ is chosen positive.
    7. *Preserving cross ratios.*

    Hierarchy of 2D Transformations

6.
7. Homography

## 3D

1. Many properties and entities of $\mathrm{P}^3$ are straightforward generalizations of those of $\mathrm{P}^2$. Example: The homogeneous coordinates of a point in $\mathrm{P}^3$ Euclidean 3-space is augmented with an extra dimension in $\mathrm{P}^2$.
    1. Note: two lines always intersect on $\mathrm{P}^2$, but not always on $\mathrm{P}^3$.
2. Points in $\mathrm{P}^3$
    1. A point in 3-space represented in homogeneous coordinates as 4-vector, i.e.
    2. $x = (x_1, x_2, x_3, x_4)^T$ with $x_4 \neq 0$
    3. represents the point $(x, y, z)^T$ of $\mathrm{R}^3$ with inhomogeneous coordinates
    4. $x = x_1/x_4$, $y = x_2/x_4$, $z = x_3/x_4$. Homogeneous points with $x_4 = 0$ represent points at infinity.
3. Projective transformation of points in $\mathrm{P}^3$:
    1. A projective transformation acting on $\mathrm{P}^3$ is a linear transformation on $x$ by a non-singular $4 \times 4$ matrix: $x' = H_{4\times4}x$.
    2. The matrix $H$ is homogeneous and has **fifteen degrees of freedom**, sixteen elements less on for scaling.
    3. As in $\mathrm{P}^2$ space, the mapping is a collineation (lines are mapped to lines, which preserves incidence relations such as intersection point of a line with a plane, and order of contact).
4. Planes in $\mathrm{P}^3$

1. A plane in 3-space may be written as:
    1. $\Pi_1 x + \Pi_2 y + \Pi_3 z + \Pi_4 = 0$
    2. Homogeneizing by $x = x_1/x_4$, $y = x_2/x_4$, $z = x_3/x_4$.
    3. $\Pi_1 x_1 + \Pi_2 x_2 + \Pi_3 x_3 + \Pi_4 x_4 = 0$
    4. $\vec{\pi}^T \vec{x} = 0$
2. Which expresses that the key point $x$ is on the plane $\vec{\pi} = (\Pi_1, \Pi_2, \Pi_3, \Pi_4)^T$.
3. Only three independent ratios $\Pi_1 : \Pi_2 : \Pi_3 : \Pi_4$ of the plane coefficients are significant, i.e. **three degrees of freedom**.
4. The first 3 components of $\Pi$ correspond to the plane normal of Euclidean geometry, i.e. $\vec{n} = (\Pi_1, \Pi_2, \Pi_3)^T$.

5. Points in $\mathrm{P}^3$
    1. Analogous to a line in $\mathrm{P}^2$, we can rewrite $\vec{\pi}^T \vec{x} = 0$ as $\vec{n} \cdot \vec{x} + d = 0$.
    2. i.e. $\begin{pmatrix} \Pi_1 \\ \Pi_2 \\ \Pi_3 \\ \Pi_4 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = 0 \Rightarrow \begin{pmatrix} \Pi_1 \\ \Pi_2 \\ \Pi_3 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} + \Pi_4$
    3. In this form $\frac{d}{\|\vec{n}\|}$ is the distance of the plane from the *origin*.
        1. Proof: using analogy to the the line in $\mathrm{P}^2$.
        2. equation of line in $\mathrm{P}^2$ in *inhomogeneous* coordinates: $\begin{pmatrix} a \\ b \end{pmatrix}^T \begin{pmatrix} x \\ y \end{pmatrix} + c = 0$. Where $\vec{n} = \begin{pmatrix} a \\ b \end{pmatrix}$ and any point on the line $\vec{v} = \begin{pmatrix} x \\ y \end{pmatrix}$.
        3. closest distance from line to the origin $d$ is the dot product of a point on the line $\vec{v}$ with the unit normal vector.
        4. $d = \frac{|\vec{v} \cdot \vec{n}|}{\|\vec{n}\|} = \frac{c}{\sqrt{a^2 + b^2}}$
        5. extending this case to the plane in $\mathrm{P}^3$ we have the distance from the plane to the origin as $\frac{d}{\|\vec{n}\|}$, where $d = \Pi_4$ and $\|\vec{n}\| = \sqrt{\Pi_1^2 + \Pi_2^2 + \Pi_3^2}$ .
    4. Again similarly, under point transformation $x' = Hx$, a plane transforms as $\boxed{\vec{\pi}' = H^{-T} \vec{\pi}}$.

6. Geometric relations between planes and points and lines:
    1. Plane defined uniquely by the *join of three points* (not collinear), or the *join of a line and point* (not incident), in general position.
    2. Two distinct planes intersect in a *unique line*.
    3. Three distinct planes intersect in a *unique point*.

7. Three points define a plane
    1. $\vec{x}_1^T \vec{\pi} = 0$
    2. $\vec{x}_2^T \vec{\pi} = 0$
    3. $\vec{x}_3^T \vec{\pi} = 0$
    4. $\begin{bmatrix} \vec{x}_1^T \\ \vec{x}_2^T \\ \vec{x}_3^T \end{bmatrix} \vec{\pi} = 0 \Rightarrow \begin{bmatrix} \vec{x}_1^T \vec{\pi} \\ \vec{x}_2^T \vec{\pi} \\ \vec{x}_3^T \vec{\pi} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$
    5. The $3 \times 4$ matrix $[\vec{x}_1, \vec{x}_2, \vec{x}_3]^T$ has rank 3 when the points are in general positions, i.e. linearly independent.
    6. The plane $\vec{\pi}$ defined by the points is obtained uniquely (up to scale) as the 1-dimensional (right null space).
    7. If the matrix $[\vec{x}_1, \vec{x}_2, \vec{x}_3]^T$ has only rank of 2, and consequently the null space is 2-dimensional.
        1. Then the points are collinear, and define a **pencil of planes** with the line of collinear points as axis.

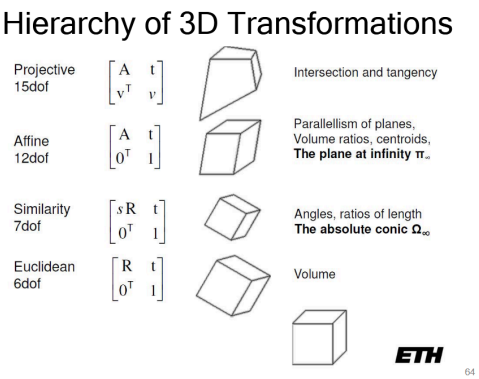
8. Three planes define a point
    1. The intersection of three unique planes $\vec{\Pi}_i$ defines a point $\vec{x}$ and can be computed as the right null space of the $3 \times 4$ matrix, compose of the planes as rows.
    2. $\begin{bmatrix} \vec{\pi}_1^T \\ \vec{\pi}_2^T \\ \vec{\pi}_3^T \end{bmatrix} \vec{x} = 0$
    3. let $A = \begin{bmatrix} \vec{\pi}_1^T \\ \vec{\pi}_2^T \\ \vec{\pi}_3^T \end{bmatrix}$, then $\begin{bmatrix} \vec{a}_1^T \\ \vec{a}_2^T \\ \vec{a}_3^T \end{bmatrix} \vec{x} = 0 \Rightarrow \begin{bmatrix} a_1^T \vec{x} \\ a_2^T \vec{x} \\ a_3^T \vec{x} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$
    4. 8 posibilities of intersection of 3 planes, one rank(A)=3, four rank(A)=2, three rank(A)=1.
    5. This development shows the duality between point and planes (point-plane duality).

9. Parametrized points on a Plane
    1. The points on the plane $\vec{\pi}$ may be written as $X$, where,

2. $X = M\vec{x}$, $X = \begin{bmatrix} \vdots & \vdots & \vdots \\ \vec{m}_1 & \vec{m}_2 & \vec{m}_3 \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$, $X = x_1\vec{m}_1 + x_2\vec{m}_2 + x_3\vec{m}_3$ (linear combination of M) describes any point in the plane.
3. The columns of the $4 \times 3$ matrix $M$ generate rank 3 null space of $\vec{\pi}^T$, i.e. $\vec{\pi}^T M = 0$, and the 3-vector $\vec{x}$ parametrizes points on the plane $\vec{\pi}$.
4. $M$ is not unique, suppose the plane is $\vec{pi} = (a, b, c, d)^T$ and $a$ is a non-zero, then $M^T$ can be written $M^T = [\vec{p}|I_{3\times 3}]$ where $\vec{p} = (-\frac{b}{a}, -\frac{c}{a}, -\frac{d}{a})^T$ is a solution to both $X = M\vec{x}$ and $\vec{\pi}^T M = 0$.
5. proof:
    1. $M^T\vec{\pi} = 0$
    2. $\begin{bmatrix} -\frac{b}{a} & 1 & 0 & 0 \\ -\frac{c}{a} & 0 & 1 & 0 \\ -\frac{d}{a} & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} -b+b \\ -c+c \\ -d+d \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

10. Lines in $\mathrm{P}^3$
    1. A line in $\mathrm{P}^3$ has **four degrees of freedom**. why 4 dof?
        1. Six dof from two points in 3D space, minus two from the invariance of translation of either of the two points along the line, leaving four.
        2. Another proof: A line may be specified by its points of intersection with two orthogonal planes. Each intersection has **two degrees of freedom**, hence 4 degrees of freedom.
    2. Awkward to represent 3D space line with a homogeneous 5-vector since it has 5 dof, we will look at two alternative representations.
    3. Suppose $\vec{A}, \vec{B}$ are two (non-incident) space points.
        1. $\vec{A}^T : [x_1, x_2, x_3, x_4]^T$
        2. $\vec{B}^T : [x_1', x_2', x_3', x_4']^T$
    4. The line joining these points (6 dofs, i.e. overparametrized) is represented by the span of the row space of the $2 \times 4$ matrix W composed of $\vec{A}^T$ and $\vec{B}^T$ as rows:
        1. $W = \begin{bmatrix} \vec{A}^T \\ \vec{B}^T \end{bmatrix}$
        2. Then:
            1. The span of $W^T$ is the **pencil of points** $\lambda\vec{A} + \mu\vec{B}$ on the line.
            2. The space of the 2D *right null space* of W is the **pencil of planes** with the line as axis.
        3. Note: It is evident that two other points, $\vec{A}'^T$ and $\vec{B}'^T$, on the same line will generate a matrix $W'$ with the same span as $W$.
        4. Hence, the representation is independent of the particular points used to define it.
        5. $W = \begin{bmatrix} \vec{A}^T \\ \vec{B}^T \end{bmatrix}$ and $W' = \begin{bmatrix} \vec{A}'^T \\ \vec{B}'^T \end{bmatrix}$ are the same.
        6. $W = \begin{bmatrix} \vec{A}^T \\ \vec{B}^T \end{bmatrix} \vec{P} = 0$
        7. Note: Suppose $\vec{P}$ and $\vec{Q}$ are the basis for the *right null space*, then $WP = 0$ and consequently $\vec{A}^T\vec{P} = \vec{B}^T\vec{P} = 0$, so that $\vec{P}$ is a plane containing the points $\vec{A}$ and $\vec{B}$.
        8. Similarly, $\vec{Q}$ is a distinct plane also containing the points $\vec{A}$ and $\vec{B}$.
        9. $\vec{A}$ and $\vec{B}$ lie on both the (linearly independent) planes $\vec{P}$ and $\vec{Q}$, so the line defined by $W$ is the plane intersection.
        10. Any **plane of the pencil**, with the line as axis, is given by the span $\lambda\vec{P} + \mu\vec{Q}$.
    5. The **dual representation** of a line as the intersection of two planes, $\vec{P}$ and $\vec{Q}$, follows in a similar manner.
        1. The line is represented as the *span (of the row space)* of the $2 \times 4$ matrix $W^*$ composed of $\vec{P}^T$ and $\vec{Q}^T$ as rows.
        2. $W^* = \begin{bmatrix} \vec{P}^T \\ \vec{Q}^T \end{bmatrix}$
        3. $W = \begin{bmatrix} \vec{A}^T \\ \vec{B}^T \end{bmatrix}$
        4. Properties:
            1. Span of $W^{*T}$ is a **pencil of planes** $\lambda\vec{P} + \mu\vec{Q}$ with the line as axis.
            2. The span of 2D null space of $W^*$ is the **pencil of points** in the line.

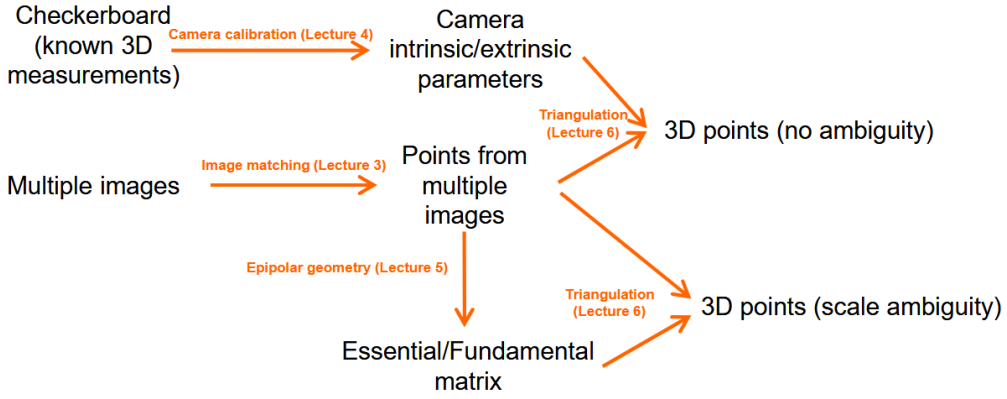
5. The two representations are related by $W^*W^T = WW^* = O_{2\times 2}$ where $O_{2\times 2}$ is a $2 \times 2$ null matrix.

11. Quadrics
    1. A quadric is a surface in $\mathrm{P}^3$ defined by the equation $\vec{x}^T Q \vec{x} = 0$.
    2. where $Q$ is a symmetric $4 \times 4$ matrix.
    3. Often the matrix $Q$ and the quadric surface it defines are not distinguished, and we simply refer to the quadric $Q$.
    4. Many properties of the quadrics follow directly from those of conics, recall $C_{3\times 3}$ 5 dof = 6 - 1, 5dof.
    5. Properties:
        1. A quadric has **9 degrees of freedom**. These correspond to the ten independent elements of a $4 \times 4$ symmetric matrix less one for scale (ten correspond to the upper triangular matrix formed in a $4 \times 4$ symmetric matrix).
        2. *Nine points in general position* define a *quadric*.
        3. If the matrix $Q$ is singular, then quadric is *degenerate*, and maybe defined by fewer points.
12. Dual Quadrics
13. 

14.

## Epipolar geometry [2]

1. All the entire pipeline of multi-view geometry theory:




2.
3. Recovering 3D geometry
    1. motivation: we don't always have a checkerboard to do camera calibration to get camera parameters (intrinsic and extrinsic), and then do triangulation to get 3D points from 2D. What we can do is apply epipolar geometry to get camera parameters from two camera images (epipolar geometry works for $n \geq 2$ images, the theory can be extended to multiple views/images). Now we concern with number of images = 2.
    2. We only have points from multiple images, then we use epipolar geometry (don't need to prepare checkerboard or 3D object). We only have points from multiple images then we apply epipolar geometry then we will get essential or fundamental matrix. Based on this essential and fundamental matrix we triangulate points from multiple images to get 3D points, but this has ambiguity up to scale.
    3. With checkerboard, we have the known 3D measurement, 3D position, actual size of checkerboard. But using epipolar geometry we do not have any known 3D measurement, it has scale ambiguity.
    4. Images from internet often do not have any calibration information. Although have scale ambiguity, the second pipeline (using epipolar geometry) is widely used and is very practical.
    5. If we know extrinsics with camera calibrations and checkerboard:
        1. e.g. you calibrate cameras with checkerboards
        2. skip all things we'll learn today

3. just triangulate (we'll learn this later) 2D points to the 3D space

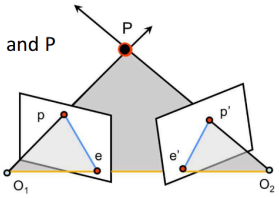
6. If we do not know extrinsics (today's focus)
    1. e.g., taking a video with your mobile phone without any calibrations/checkerboards.
    2. Epipolar geometry!
    3. We get essential/fundamental matrices
    4. We do not have checkerboard, which provides actual 3D measurements -> we get extrinsics up to scale.

4. Epipolar Geometry (Diagram):
    1. Baseline
        1. The line between the two camera centers O_1 and O_2.
    2. Epipolar plane
        1. Defined by P, O_1, and O_2; contains baseline and P
    3. Epipoles (e and e')
        1. intersection of baseline and image plane
        2. Projection of the other camera center
    4. Epipolar lines
        1. intersection of epipolar plane with the image plane

5. Epipolar constraint:

## Epipolar Geometry

- Baseline (Yellow line)
  – The line between the two camera centers $O_1$ and $O_2$
- Epipolar plane (gray plane)
  – Defined by P, $O_1$, and $O_2$; contains baseline and P
- Epipoles (e and e')
  – ∩ of baseline and image plane
  – Projection of the other camera center
- Epipolar lines (Blue lines)
  – ∩ of epipolar plane with the image plane




1.
2. The relationship between corresponding image points
    1. The world reference system aligned with the left camera (assumption).
    2. The right camera has orientation $R$ and offset $t$.
    3. Left camera projection matrices:
        - $M = K\,[I \quad 0]$
        - $\vec{p} = M\vec{P} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$
    4. Right camera projection matrices:
        - $M' = K'\,[R \quad \vec{t}]$
        - $\vec{p'} = M'\vec{P} = \begin{bmatrix} u' \\ v' \\ 1 \end{bmatrix}$
3. Derivation of essential matrix:
    1. Using normalized coordinates, $K = K' = I$
    2. $\mathbf{p}'$ wrt. $\mathbf{O}_2$ : $\mathbf{p}' - \mathbf{t}$
    3. $\mathbf{p}'$ wrt. $\mathbf{O}_2$ in terms of (expressed in [4]) $\mathbf{O}_1$: $R^T(\mathbf{p}' - \mathbf{t})$
    4. $\mathbf{O}_2$ wrt. $\mathbf{O}_1$: $\mathbf{t}$
    5. $\mathbf{O}_1$ wrt. $\mathbf{O}_2$: $-\mathbf{t}$
    6. $\mathbf{O}_1$ wrt. $\mathbf{O}_2$ in terms of (expressed in) $\mathbf{O}_1$: $-R^T\mathbf{t}$
    7. $R^T(\mathbf{p}' - \mathbf{t}) \times (-R\mathbf{t}) =$
    8. $R^T\mathbf{t} \times R^T(\mathbf{p}' - \mathbf{t})$
    9. $= R^T[t \times (\mathbf{p}' - \mathbf{t})]$
    10. $= R^T(\mathbf{t} \times \mathbf{p}') = \hat{\mathbf{n}}$
    11. Now, $\vec{O_1P}$ dot product with normal to the epipolar plane is zero, hence:
    12. $\hat{\mathbf{n}}^T\mathbf{p} = 0$

13. $[R^T(\mathbf{t} \times \mathbf{p}')]^T\mathbf{p} = 0$
14. $(\mathbf{t} \times \mathbf{p}')^T R\mathbf{p} = 0$
15. $([\mathbf{t}\times]\mathbf{p}')^T R\mathbf{p} = 0$
16. $\mathbf{p}'^T[\mathbf{t}\times]^T R\mathbf{p} = 0$
17. but because $[\mathbf{t}\times]$ is skew-symmetric, $[\mathbf{t}\times]^T = -[\mathbf{t}\times]$
18. $\mathbf{p}'[\mathbf{t}\times]R\mathbf{p} = 0$
19. $\mathbf{p}'E\mathbf{p} = 0$
20. $E = [\mathbf{t}\times]R$

4. Properties of Essential matrix
    1. Establish constraints between matching image points.
    2. Determine relative position and orientation of two cameras.
    3. **5 degrees of freedom** (R:3,$\mathbf{t}$:3), but scale is not known.
    4. We can multiply any scalar $\lambda$, we cannot uniquely determine essential matrix $E$. So $E$ has 5 dof. 6-1, -1 from scale ambiguity (we can't uniquely determine E.
    5. $\lambda\mathbf{p}'^T E\mathbf{p} = 0 \Rightarrow \mathbf{p}'(\lambda E)\mathbf{p} = 0$
    6. E, minimally necessary but not perfect information that's why we call this as *essential matrix*.
5. How to generalize Essential matrix? Fundamental matrix.
    1. $\mathbf{p} = M\mathbf{P} = K\begin{bmatrix} I & 0 \end{bmatrix}\mathbf{P}$. $K$ is the camera intrinsic matrix and $\begin{bmatrix} I & 0 \end{bmatrix}$ is the result of the canonical projection matrix $\Pi$ times the camera extrinsic. i.e. $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}\begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$. But in this case $R = \text{Identity}$, $\mathbf{t} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$. So, we essentially throw out the last row of $\begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$ to get $\begin{bmatrix} R & \mathbf{t} \end{bmatrix} = \begin{bmatrix} I & 0 \end{bmatrix}$.
    2. $\mathbf{p}' = M'\mathbf{P} = K'\begin{bmatrix} R & \mathbf{t} \end{bmatrix}\mathbf{P}$. Similar to the first camera, however, in this second camera case R is not the identity matrix and $\mathbf{t}$ is nonzero. Therefore, the matrix multiplication of the canonical projection matrix and the camera extrinsic matrix will yield $\begin{bmatrix} R & \mathbf{t} \end{bmatrix}$.
    3. $\mathbf{p} \rightarrow K^{-1}\mathbf{p}$
    4. $\mathbf{p}' \rightarrow K'^{-1}\mathbf{p}'$
    5. $\mathbf{p}'^T K'^{-T} E K^{-1}\mathbf{p} = 0$
    6. $F = K'^{-T}[\mathbf{t}_\times]RK^{-1}$
6. Deriving the epipolar constraint:
    1. Project a 3D point $\mathbf{X}$ with the two camera matrices:

    $$\mathbf{x} = P\mathbf{X}, \qquad \mathbf{x}' = P'\mathbf{X},$$

    where $P = K[I \mid 0]$ and $P' = K'[R \mid t]$.
    2. The camera centers $\mathbf{C}_1, \mathbf{C}_2$ and $\mathbf{X}$ define a single epipolar plane. A normal to that plane is $\mathbf{t} \times (R\mathbf{X})$.
    3. Represent the cross product with the skew-symmetric matrix $[\mathbf{t}]_\times$ so that

    $$\mathbf{t} \times (R\mathbf{X}) = [\mathbf{t}]_\times R\mathbf{X}.$$

    4. Every vector inside the plane is orthogonal to this normal, hence

    $$(R\mathbf{X})^\top [\mathbf{t}]_\times \mathbf{X} = 0.$$

    5. Substitute $\mathbf{X} \propto K^{-1}\mathbf{x}$ and $R\mathbf{X} + \mathbf{t} \propto K'^{-1}\mathbf{x}'$ and remove scale factors to obtain

    $$\mathbf{x}'^\top (K'^{-\top}[\mathbf{t}]_\times RK^{-1})\mathbf{x} = 0.$$

    Define the matrix in parentheses as the **fundamental matrix**

    $$F = K'^{-\top}[\mathbf{t}]_\times RK^{-1}.$$

    Therefore every correspondence satisfies the epipolar constraint

    $$\mathbf{x}'^\top F\mathbf{x} = 0.$$

    Because the constraint is linear in the entries of $F$, we can stack equations from multiple matches to build $A\mathbf{f} = 0$ for the eight-point solver.
7. Essential matrix vs Fundamental matrix
    1. Similarity
        1. Both relate the matching points

        1. Encode epipolar geometry of two views & camera parameters.
    2. Differences
        1. E (Essential matrix) encodes only the camera extrinsic parameter.
        2. F (fundamental matrix) also encodes the intrinsic parameters.
        3. E assumes known intrinsics, lives in camera coordinates, and decomposes into relative rotation and translation direction.
        4. F works in pixel coordinates, needs not intrinsics, maps $\mathbf{p}$ or $\mathbf{x}$ to its epipolar line $l' = F\mathbf{x}$.
8. Properties of the fundamental matrix
    1. 3 by 3
    2. homogeneous (has scale ambiguity)
    3. **rank(F)=2**
        1. The potential matching point point is located on a line.
    4. $F$ has **7 degrees of freedom** (3x3 -1 (rank2) -1 (scale ambiguity)=7)
9. How can we use the fundamental matrix?
    1. Given a 2D point in one image and fundamental matrix, we can get epipolar line from the other image (candidate positions of matched points).
    2. Image matching accuracy becomes much better using the fundamental matrix (get rid of all the false positive matching).
    3. Without knowing camera intrinsic and extrinsic parameters (without requiring camera parameters, without using checkerboard).
    4. If we know $\mathbf{p}$, epipolar line is $l = F\mathbf{p}$
    5. If we know $\mathbf{p}'$, epipolar line is $l' = F^T\mathbf{p}$
    6. With the epipolar line, we can reduce the matching space
        1. Not entire image, only search points on th epipolar line.
        2. Find matched points and lift them to the 3D space with triangulation.
    7. $\mathbf{p}'F\mathbf{p} = 0$, $F = K'^{-T}[\mathbf{t}_\times]RK^{-1}$
10. Recovering Fundamental Matrix
    1. How to recover F?
    2. 8-point pairs required
        1. Each point pair gives one equation
        2. F is known up to scale
            1. The linear system is homogeneous
    3. The problem: two views of the same scene
        1. Same scene, two cameras, different positions.
            1. The same 3D points appear at different pixels in each image.
            2. SIFT finds pairs: $x$ in image 1 $\leftrightarrow$ $x'$ in image 2.
    4. The question
        1. Without calibration, what geometry links these views?
        2. If we know $x$ in image 1, where must $x'$ appear?
    5. The answer
        1. $F$ maps any $x$ in image 1 to a line in image 2 -- The epipolar line. The true match $x'$ must lie on it.
    6. Computing $F$ from $N \geq 8$ point pairs -- no calibration required -- is the 8-point algorithm.
    7. Applications:
        5. Fundamental matrix encodes all of this.
        1. Stereo depth estimation
        2. 3D reconstruction
        3. Visual odometry
        4. Structure-from-motion
    8. What is F?
        1. $3 \times 3$ matrix
            1. Rank 2, 7 degrees of freedom
        2. Maps a point to a line
            1. $x$ in image 1 $\rightarrow$ epipolar line $l'$ in image 2
        3. Correspondence constraint

    1. Any true match $x'$ must lie on $l'$
4. Enables stereo matching
    1. 2D search becomes 1D search along a line
5. $l' = Fx$, (point $\rightarrow$ epipolar line)
6. $x'^T Fx = 0$, (epipolar constraint)

9. The core equation
    1. $x'Fx = 0$
    2. with $x = [u, v, 1]^T$ and $x' = [u', v', 1]^T$, expanding gives:

$$u'u \cdot f_{11} + u'v \cdot f_{12} + u' \cdot f_{13} + v'u \cdot f_{21} + v'v \cdot f_{22} + v' \cdot f_{23} + u \cdot f_{31} + v \cdot f_{32} + f_{33} = 0$$

    3. Key insight:
        1. Coordinates $(u, v, u', v')$ come from feature matches and are known.
        2. The 9 entries of F are the unknowns and this equation is **linear** in them.
        3. That means we can solve for F with straightforward linear algebra.
    4. Why 8 points?
        1. Because F has 7 degrees of freedom.
        2. Technically 7 pairs suffice, but the 7-point method requires solving a cubic equation with up to 3 solutions.
    5. Building the matrix equation $Af = 0$
        1. Each point correspondence (x, x') contributes to one row of A:
        2. $a_i^T = [u'u \quad u'v \quad u' \quad v'u \quad v'v \quad v' \quad u \quad v \quad 1]$
        3. Stack $f = vec(F) = [f_{11} \quad f_{12} \quad f_{13} \quad f_{21} \quad f_{22} \quad f_{23} \quad f_{31} \quad f_{32} \quad f_{33}]^T$, 9 unknowns.
        4. A ($N \times 9$)

$$A = \begin{bmatrix} a_{\text{pair1}}^T \\ a_{\text{pair2}}^T \\ \vdots \\ a_{\text{pair}N}^T \end{bmatrix}$$

        5. Goal: find non-trivial solution $f$ such that $Af \approx 0$.
        6. Solution: use SVD. $A = U\Sigma V^T$, U: ($N \times N$) orthogonal, $\Sigma$ : ($N \times 9$) diagonal $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_9 \geq 0$, V: ( $9 \times 9$) orthogonal. The last column of V (right singular vector for smallest $\sigma_9$) or last row of $V^T$ minimizes $||Af||$ subject to $||f|| = 1$.
        7. Reshape this 9-vector back into $3 \times 3$ initial estimate of $F$.
        8. Noise-free if $\sigma_9 = 0$ (exact solution), noisy data: $\sigma_9$ is small, still last column of $V$ is the best estimate.
        9. Enforcing rank(F)=2
            1. Problem: SVD gives $\tilde{F}$ which may be rank 3. True $F$ requires rank 2 -- all epipolar lines converge at the epipole, so $\det(F) = 0$.
            2. Fix: apply a second SVD
                1. Take SVD of $\tilde{F} = U \cdot diag(\sigma_1, \sigma_2, \sigma_3) \cdot V^T$.
                2. Zero out the smallest singular value: $F^* = U \cdot diag(\sigma_1, \sigma_2, 0) \cdot V^T$.
                3. $F^*$ is the nearest rank-2 matrix to our initial estimate $\tilde{F}$ in the Frobenius norm sense. Now $\det(F)$ exactly 0 as required.
            3. This fix fulfilled:
                1. Nearest rank-2 matrix (Frobenius norm).
                2. $\det(F^*) = 0$ as required.
                3. Epipolar lines now all pas through $e'$.

| 7-point algorithm | 8-point algorithm |
| --- | --- |
| 1. Exactly 7 correspondences | 8+ correspondences |
| 2. Solves a cubic equation | Fully linear -- solved with SVD |
| 3. Up to 3 solutions | Single unique solution |
| 4. More complex to implement | Simple works well with RANSAC |

    10. The normalized 8-point algorithm

1. Problem: pixel coords (0-1920) make a ill-conditioned SVD, results are unreliable.
2. Hartley's fix, Related Normalization_Techniques:
    1. Translate centroid of all points to origin
    2. Scale so mean distance from origin=$\sqrt{2}$
    3. Apply independently per image (T and T')
    4. $x_{DN} = Tx$, $x'_{DN} = T'x'$, DN: data normalized (normalize)
    5. Solve for $F_{DN}$, then $F^* = T'^T F_{DN} T$ (unnormalize)
    6. Always normalize, Hartley (1997) showed accuracy improves by orders of magnitude
3. The normalized 8-point algorithm
    1. Summary:
        1. Normalize: Translate centroids to origin, scale to mean dist: $\sqrt{2}$. Get $T$, $T'$.
        2. Build A: Fill $N \times 9$ matrix, one row per correspondence using $x_{DN}$,$x'_{DN}$.
        3. SVD of A: $A = U\Sigma V^T \rightarrow f = \text{last col of V} \rightarrow \text{reshape to } 3 \times 3 = \tilde{F}$
        4. **Enforce rank 2**: SVD of $\tilde{F} \rightarrow$ set $\sigma_3 = 0 \rightarrow F_{DN} = U \cdot diag(\sigma_1, \sigma_2, 0) \cdot V^T$.
        5. Denormalize: $F^* = T'^T F_{DN} T$, this is our Fundamental matrix.
    2. In practice wrap this entire pipeline in RANSAC.
        1. For example in visual odometry, in each iteration: [7]
            1. randomly choose 8 point correspondences from all point correspondences.
            2. compute fundamental matrix $F$ from this 8 points.
            3. compute the number of inliers: search all $N$ point correspondence, keep those point correspondences which satisfy $\tilde{x}'^T F \tilde{x} \leq \text{threshold}$.
            4. choose the $F$ with max number of inliers.
            5. Solve camera pose from Essential matrix
                1. Take SVD for essential matrix $E$, $E = U\Sigma V^T$. The essential matrix must have two singular values which are equal and another which is zero,
                2. $Sigma = \begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 0 \end{pmatrix}$
                3. Define matrix: $W = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$,
                4. $W^{-1} = W^T = \begin{pmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$.
                5. Translation $t$ and rotation $R$ can be computed as:
                6. $[t]_\times = UW\Sigma U^T$
                7. $R = UW^{-1}V^T$.
    3. Limitations:
        1. Sensitive to outliers -- always use RANSAC
        2. Fails in degenerate configurations (coplanar, pure rotation)
        3. Requires $\geq$ 8 correspondences, more is better with noise
        4. Estimates $F$ up to scale only
10. Code: epipolar.py

```python
# -*- coding: utf-8 -*-
"""
Created on Sat Apr 25 16:28:09 2026

@author: reina
"""

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D


def homogenize(x):
    # x is of size (d, n)
    # d - dimension
    # n - number of points
    # Converts points from inhomogeneous to homogeneous coordinates
    return np.vstack((x, np.ones((1, x.shape[1]))))
```

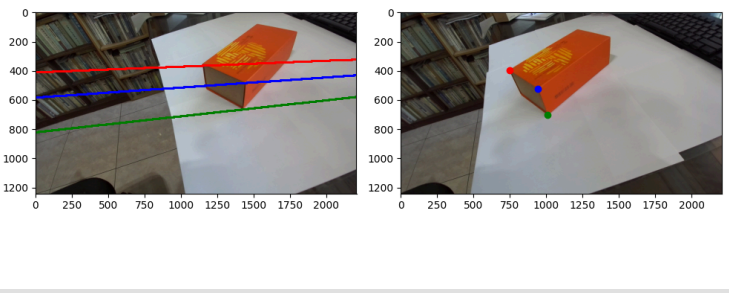
```python
def dehomogenize(x):
    # x is of size (d+1, n)
    # d - dimension
    # n - number of points
    # Converts points from homogeneous to inhomogeneous coordinates
    return x[:-1, :] / x[-1, :]


def data_normalization(pts):
    """
    Data normalization of n dimensional pts

    Input:
        pts - in inhomogeneous coordinates
    Output:
        pts - data normalized points
        T - corresponing transformation matrix
    """
    # pts: (d, n)
    pts_hom = homogenize(pts)
    pts_mean = np.mean(pts_hom, axis=1)
    pts_var = np.var(pts_hom, axis=1)
    # d==2
    T = np.zeros((3, 3))
    var_x = pts_var[0]
    var_y = pts_var[1]
    mean_x = pts_mean[0]
    mean_y = pts_mean[1]
    s = np.sqrt(2 / (var_x + var_y))
    T[0, 0] = s
    T[0, 2] = -s * mean_x
    T[1, 1] = s
    T[1, 2] = -s * mean_y
    T[2, 2] = 1
    pts = T @ pts_hom
    return pts, T


def compute_fundamental_matrix(pts1_normal, pts2_normal, T1, T2):
    # pts in inhomogeneous coordinates
    # pts1_normal: (d, n), d = 2
    # pts2_normal: (d, n), d = 2
    n = pts1_normal.shape[1]
    A = np.zeros((n, 9), dtype=np.float64)
    for i in range(n):
        u, v = pts1_normal[:, i]
        u_, v_ = pts2_normal[:, i]
        A[i, :] = np.array([u_ * u, u_ * v, u_, v_ * u, v_ * v, v_, u, v, 1])
    u, s, vh = np.linalg.svd(A)
    F = vh[-1, :].T.reshape(3, 3)
    print(F.shape)
    u_, s_, vh_ = np.linalg.svd(F)
    s_[2] = 0.0
    F_normal = u_ @ np.diag(s_) @ vh_
    # Un-normalize F
    F_unnormal = T2.T @ F_normal @ T1
    return F_unnormal


def compute_epipolar_error(F, pts1, pts2):
    # pts1: (d, n), n=2
    # pts2: (d, n), n=2
    pts1_hom = homogenize(pts1)  # (d+1, n)
    pts2_hom = homogenize(pts2)  # (d+1, n)
    error = np.sum(pts2_hom.T @ F @ pts1_hom)
    return error


if __name__ == "__main__":
    img1 = plt.imread("box_left_2k.jpg")
    img2 = plt.imread("box_right_2k.jpg")
    print(img1.shape)

    pts1 = np.array(
        [
            [1148, 351],
            [1441, 481],
            [1427, 662],
            [1172, 525],
            [1524, 80],
            [1795, 134],
            [1744, 307],
            [1564, 320],
            [1579, 306],
            [1584, 288],
        ]
    )  # x: (n, d), left image points

    pts2 = np.array(
        [
```

```
            [747, 397],
            [944, 522],
            [1010, 704],
            [822, 558],
            [1212, 116],
            [1440, 170],
            [1433, 350],
            [1131, 360],
            [1152, 342],
            [1166, 326],
        ]
    )  # x': (n, d), right image points

    K1 = np.array([[1062.04, 0, 1098.52], [0, 1061.74, 599.665], [0, 0, 1]])
    K2 = np.array([[1060.59, 0, 1127.03], [0, 1060.06, 645.711], [0, 0, 1]])

    # TODO 1 (15 points): Normalize image points.
    #       There is no need to use for loops in this task. If you use for loops in this task, you can get
at most 10 point.

    pts1_normal_hom, T1 = data_normalization(pts1.T)
    pts2_normal_hom, T2 = data_normalization(pts2.T)
    print(np.mean(np.linalg.norm(dehomogenize(pts1_normal_hom), axis=0)))

    # TODO 2 (25 points): Compute the fundamental matrix F.

    pts1_normal = dehomogenize(pts1_normal_hom)
    pts2_normal = dehomogenize(pts2_normal_hom)
    F = compute_fundamental_matrix(pts1_normal, pts2_normal, T1, T2)
    print("Fundamental matrix, F: ", F)
    error = compute_epipolar_error(F, pts1.T, pts2.T)
    print("epipolar error: ", error)

    # TODO 3 (15 points): Draw epipolar lines in image1.
    pts_left_hom = homogenize(pts1.T)
    pts_right_hom = homogenize(pts2.T)
    print(pts_left_hom.shape)  # (3, 10)
    print(pts_right_hom.shape)  # (3, 10)
    print(F.shape)  # (3, 3)

    l_left = F.T @ pts_right_hom
    l_right = F @ pts_left_hom

    print("l_left.shape", l_left.shape)  # (3, 10)
    print("l_right.shape", l_right.shape)  # (3, 10)
    marker = ["ro--", "bo--", "go--"]
    fig = plt.figure(figsize=(10, 5))
    ax1 = fig.add_subplot(121)
    # Plot first three epipolar lines in left image
    for i in range(3):
        lx, ly, lz = l_left[:, i]
        for j in range(img1.shape[1]):
            # from ax + by + c = lx * x + ly * y + lz * 1 = 0
            x = int(j)
            y = int(-(lx / ly) * x - (lz / ly))
            # print('x, y', x, y)
            ax1.plot(x, y, marker[i], linewidth=1, markersize=1)

    ax1.imshow(img1)

    ax2 = fig.add_subplot(122)
    ax2.plot(pts2[0, 0], pts2[0, 1], "ro")
    ax2.plot(pts2[1, 0], pts2[1, 1], "bo")
    ax2.plot(pts2[2, 0], pts2[2, 1], "go")
    ax2.imshow(img2)
    plt.tight_layout()
    plt.savefig("plot_epipolar_lines.png")
    plt.show()

    # TODO 4 (10 points): Draw the epipole and epipolar lines in image1.
    # Compute epipole (point)
    u, s, vh = np.linalg.svd(F)
    e_hom = vh[-1, :].T
    e = (e_hom[:2] / e_hom[-1]).astype(int)
    fig = plt.figure(figsize=(10, 5))
    ax3 = fig.add_subplot(111)
    ax3.plot(e[0], e[1], "kx")
    for i in range(3):
        lx, ly, lz = l_left[:, i]
        for j in range(e[0]):
            # from ax + by + c = lx * x + ly * y + lz * 1 = 0
            x = int(j)
            y = int(-(lx / ly) * x - (lz / ly))
            # print('x, y', x, y)
            ax3.plot(x, y, marker[i], linewidth=1, markersize=1)
    img1_copy = np.zeros((img1.shape[0], e[0] + 50, 3), dtype=np.uint8)
    img1_copy.fill(255)
    img1_copy[: img1.shape[0], : img1.shape[1], :] = (
        img1  # img[y, x], indexing in image
    )
```

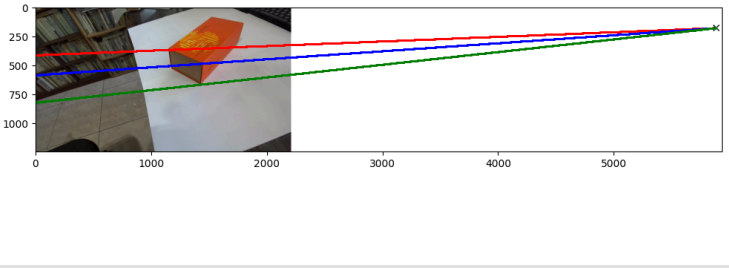
```
    ax3.imshow(img1_copy)
    plt.tight_layout()
    plt.savefig("plot_epipole.png")
    plt.show()
    # TODO 5 (5 points): Compute the 4 possible camera matrices.
    # Compute essential matrix
    E = K2.T @ F @ K1
    # Perform SVD on the essential matrix
    U, S, Vh = np.linalg.svd(E)  # Vh is V transpose
    # Let u3 be the last column of U
    u3 = U[:, -1]
    # Define W=R_{90}, rotates u1 and u2 by 90^o
    W = np.array([[0, -1, 0], [1, 0, 0], [0, 0, 1]])
    if np.linalg.det(U @ W @ Vh) < 0:
        W = -W
    # Suppose the first camera matrix is P=[I, 0].
    # Then there are four possible choices for the second camera matrix P'=[R, t]
    R1 = U @ W @ Vh
    R2 = U @ W.T @ Vh
    t1 = u3
    t2 = -u3
    print(R1.shape)
    print(t1.shape)
    Rt_list = [
        np.hstack((R1, t1[..., np.newaxis])),
        np.hstack((R1, t2[..., np.newaxis])),
        np.hstack((R2, t1[..., np.newaxis])),
        np.hstack((R2, t2[..., np.newaxis])),
    ]
    # TODO 6 (25 points): Triangulation.
    #         There is no need to use for loops when computing the reconstruction errors of pts1 (or pts2).
    #         If you do, you can get at most 20 point.
    X_list = []
    P1 = np.zeros((3, 4), dtype=np.float64)
    P1[:, :3] = K1
    for j in range(len(Rt_list)):
        P2 = K2 @ Rt_list[j]
        X = np.zeros((pts1.shape[0], 3), dtype=np.float64)
        for i in range(pts1.shape[0]):
            A = np.zeros((4, 3), dtype=np.float64)
            b = np.zeros((4, 1), dtype=np.float64)
            xi, yi = pts1[i, 0], pts1[i, 1]
            xi_, yi_ = pts2[i, 0], pts2[i, 1]
            p11 = P1[0, 0]
            p11_ = P2[0, 0]
            p12 = P1[0, 1]
            p12_ = P2[0, 1]
            p13 = P1[0, 2]
            p13_ = P2[0, 2]
            p14 = P1[0, 3]
            p14_ = P2[0, 3]
            p21 = P1[1, 0]
            p21_ = P2[1, 0]
            p22 = P1[1, 1]
            p22_ = P2[1, 1]
            p23 = P1[1, 2]
            p23_ = P2[1, 2]
            p24 = P1[1, 3]
            p24_ = P2[1, 3]
            p31 = P1[2, 0]
            p31_ = P2[2, 0]
            p32 = P1[2, 1]
            p32_ = P2[2, 1]
            p33 = P1[2, 2]
            p33_ = P2[2, 2]
            p34 = P1[2, 3]
            p34_ = P2[2, 3]
            # Construct A matrix
            A11 = p11 - p31 * xi
            A12 = p12 - p32 * xi
            A13 = p13 - p33 * xi
            A21 = p21 - p31 * yi
            A22 = p22 - p32 * yi
            A23 = p23 - p33 * yi
            A31 = p11_ - p31_ * xi_
            A32 = p12_ - p32_ * xi_
            A33 = p13_ - p33_ * xi_
            A41 = p21_ - p31_ * yi_
            A42 = p22_ - p32_ * yi_
            A43 = p23_ - p33_ * yi_
            A[0, 0] = A11
            A[0, 1] = A12
            A[0, 2] = A13
            A[1, 0] = A21
            A[1, 1] = A22
            A[1, 2] = A23
            A[2, 0] = A31
            A[2, 1] = A32
            A[2, 2] = A33
            A[3, 0] = A41
```

```
            A[3, 1] = A42
            A[3, 2] = A43
            b[0, 0] = p34 * xi - p14
            b[1, 0] = p34 * yi - p24
            b[2, 0] = p34_ * xi_ - p14_
            b[3, 0] = p34_ * yi_ - p24_
            # Solve for [Xi, Yi, Zi].T using Ax=b
            m = np.linalg.lstsq(A, b)[0]
            X[i, :] = m.squeeze()

        X_list.append(X)

    j_best = [j for j in range(len(X_list)) if np.all(X_list[j][:, 2] > 0)]

    print("j_best", j_best)
    M = X_list[j_best[0]]  # (n, d) = (n, 3), estimated 3D points
    m1 = P1 @ homogenize(M.T)  # reprojected 2D points in image1
    P2 = K2 @ Rt_list[j_best[0]]

    m2 = P2 @ homogenize(M.T)  # reprojected 2D points in image2
    # print(m1)  # (d+1, n)
    # print(m2)  # (d+1, n)
    m1_dehom = dehomogenize(m1)
    m2_dehom = dehomogenize(m2)
    reproj_error1 = np.sum(np.sqrt(np.mean((m1_dehom - pts1.T) ** 2, axis=1)))
    reproj_error2 = np.sum(np.sqrt(np.mean((m2_dehom - pts2.T) ** 2, axis=1)))

    print(reproj_error1)
    print(reproj_error2)

    # TODO 7 (5 points): Visualize the 3D reconstruction.
    x = M[:, 0]
    y = M[:, 1]
    z = M[:, 2]

    fig = plt.figure()
    ax1 = fig.add_subplot(121, projection="3d")
    ax1.view_init(elev=-90, azim=-120)
    ax1.set_aspect("equal")
    ax1.scatter(x, y, z, s=1.7)
    vertices_pairs = [
        (0, 1),
        (1, 2),
        (2, 3),
        (3, 0),
        (4, 5),
        (5, 6),
        (0, 4),
        (1, 5),
        (2, 6),
    ]
    # Plot points using plt.plot(x's, y's, z's)
    for i in range(len(vertices_pairs)):
        v_i = vertices_pairs[i][0]
        v_j = vertices_pairs[i][1]
        plt.plot(
            [M[v_i, 0], M[v_j, 0]],
            [M[v_i, 1], M[v_j, 1]],
            [M[v_i, 2], M[v_j, 2]],
            "ro-",
        )

    ax2 = fig.add_subplot(122, projection="3d")
    ax2.view_init(elev=-139, azim=-113)
    ax2.set_aspect("equal")
    ax2.scatter(x, y, z, s=1.7)

    # Plot points using plt.plot(x's, y's, z's)
    vertices_pairs = [
        (0, 1),
        (1, 2),
        (2, 3),
        (3, 0),
        (4, 5),
        (5, 6),
        (0, 4),
        (1, 5),
        (2, 6),
    ]
    for i in range(len(vertices_pairs)):
        v_i = vertices_pairs[i][0]
        v_j = vertices_pairs[i][1]
        plt.plot(
            [M[v_i, 0], M[v_j, 0]],
            [M[v_i, 1], M[v_j, 1]],
            [M[v_i, 2], M[v_j, 2]],
            "ro-",
        )

    plt.savefig("triangulated_3D_points.png")
```

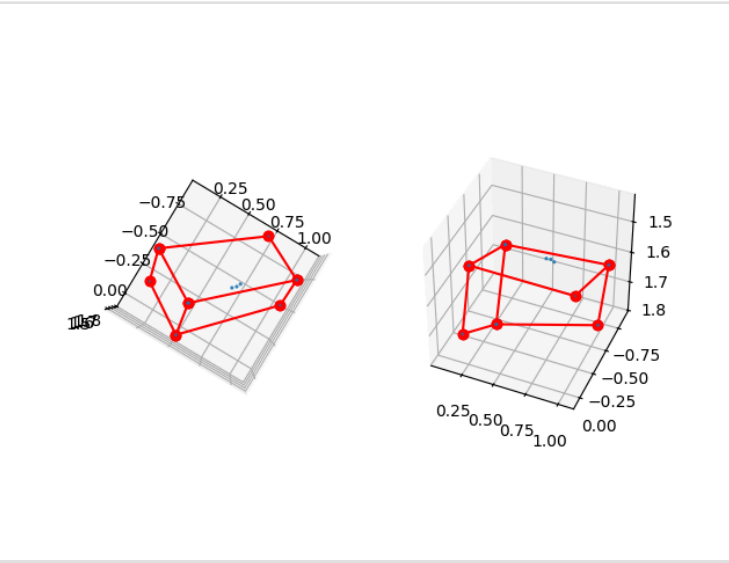
```
    plt.show()
```

11. 


Epipolar lines

12. 


epipole & epipolar lines

13. 

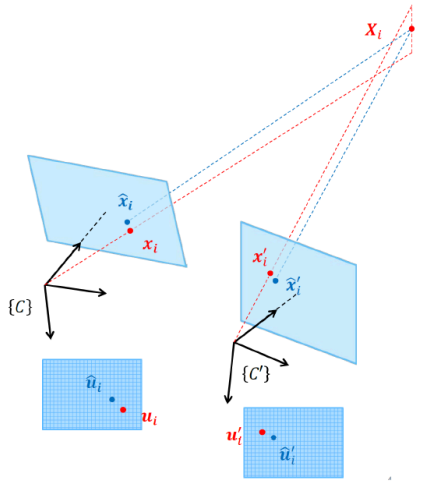

triangulated points

11. E vs F [5]
    1. If we estimate F in each RANSAC iteration, then we need $N = 8$ correspondences to determine F.
    2. If instead E is determined, it is sufficient with N=5. correspondences .
    3. If the internal calibration K is known, we can reduce $r$=number of RANSAC iterations, by using E instead of F.

## Triangulation [8]

1. Triangulation is estimating a 3D point $\mathbf{X}_i$ for a noisy 2D correspondence under the assumption that camera matrices $P$ and $P'$ are known.
2. Introduction
    1. We have seen that two perspective cameras observing the same points must satisfy the epipolar constraint.
    2. The essential matrix $E = [t]_\times R$.

3. $\tilde{\mathbf{x}}'^T E\tilde{\mathbf{x}} = 0$.
4. The fundamental matrix $F = K'^{-T}EK^{-1}$.
5. $\tilde{\mathbf{u}}'^T F\tilde{\mathbf{u}} = 0$
6. Being observed by two perspective cameras also puts a strong geometric constraint on the observed points $\mathbf{X}_i$.
7. $$P\mathbf{X}_i = \mathbf{u}_i$$ $$P'\mathbf{X}_i = \mathbf{u}'_i$$
8. Assume that we know the camera matrices $P$. $P'$ and 2D correspondences $\mathbf{u}_i \leftrightarrow \mathbf{u}'_i$.
9. In order to determine the 3D point $\mathbf{X}_i$ it is tempting to inverse project the two image points and determine their intersection.
10. But due to noise, the two rays in 3D will 'never' truly intersect, so we need to estimate a best solution to the problem.
11. Several ways to approach the problem depending on what we choose to optimize over
    1. only errors in $\mathbf{u}_i \leftrightarrow \mathbf{u}'_i$?
    2. errors in $\mathbf{u}_i \leftrightarrow \mathbf{u}'_i$, $P$ and $P'$?

3. The 3D midpoint: minimizing the 3D error
    1. 

    2. One natural estimate for $\mathbf{X}_o$ is the midpoint on the shortest line between to two inverse projected rays.
    3. This minimize the 3D error, but typically not the reprojection error
        1. $$\epsilon_i = d(\mathbf{u}_i, P\mathbf{X}_i)^2 + d(\mathbf{u}'_i, P'\mathbf{X}_i)^2$$ $$\epsilon_i = d(\mathbf{u}_i, \hat{\mathbf{u}}_i)^2 + d(\mathbf{u}'_i, \hat{\mathbf{u}}'_i)^2$$
    4. The difference becomes clear when $\mathbf{X}_i$ is much closer to one of the cameras than the other.

4. Linear triangulation: Minimizing the algebraic error [8], [9]
    1. this algorithm uses the two equations for perspective projection to solve for the 3D point that are optimal in a least squares sense.
    2. Each perspective camera model gives rise to two equations on the three entries of $\mathbf{X}_i$.
    3. Can we compute $\mathbf{X}$ from two correspondences $mathbfx$ and $\mathbf{x}'$? yes if perfect measurements.
    4. need to find the best fit
    5. since there will not be a point that satisfies both constraints because the measurements are usually noisy.
    6. apply DLT: Remove scale factor (from homogeneous coordinates), convert to linear system and solve with SVD.
    7. $\tilde{\mathbf{u}}_i = P\tilde{\mathbf{X}}_i$
    8. same ray direction, but differs by a scale factor.
    9. $\tilde{\mathbf{x}}_i \times P\tilde{\mathbf{X}}_i = 0$
    10. $$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} \times \begin{bmatrix} \mathbf{p}_1^T\tilde{\mathbf{X}}_i \\ \mathbf{p}_2^T\tilde{\mathbf{X}}_i \\ \mathbf{p}_3^T\tilde{\mathbf{X}}_i \end{bmatrix} = 0$$
    11. $$\begin{bmatrix} v_i\mathbf{p}_3^T - \mathbf{p}_2^T \\ \mathbf{p}_1^T - u_i\mathbf{p}_3^T \\ u_i\mathbf{p}_2^T - v_i\mathbf{p}_1^T \end{bmatrix} \tilde{\mathbf{X}}_i = 0$$
    12. Third line is a linear combination of the first and second lines ($u_i$ times the first line plus $v_i$ times the second line).
    13. $\begin{bmatrix} v_i\mathbf{p}_3^T - \mathbf{p}_2^T \\ \mathbf{p}_1^T - u_i\mathbf{p}_3^T \end{bmatrix} \tilde{\mathbf{X}}_i = 0$ (taking the first and second line).

14. $\begin{bmatrix} v_i\mathbf{p}_3^T - \mathbf{p}_2^T \\ u_i\mathbf{p}_3^T - \mathbf{p}_1^T \end{bmatrix}\tilde{\mathbf{X}}_i = 0$ (multiply second row by -1, because RHS is 0).
15. Since we have two views: $\begin{bmatrix} v_i\mathbf{p}_3^T - \mathbf{p}_2^T \\ u_i\mathbf{p}_3^T - \mathbf{p}_1^T \\ {v'}_i\mathbf{p'}_3^T - \mathbf{p'}_2^T \\ {u'}_i\mathbf{p'}_3^T - \mathbf{p'}_1^T \end{bmatrix}\tilde{\mathbf{X}}_i = 0$
16. $$\begin{bmatrix} v_ip_{31} - p_{21} & v_ip_{32} - p_{22} & v_ip_{33} - p_{23} & v_ip_{34} - p_{24} \\ u_ip_{31} - p_{11} & u_ip_{32} - p_{12} & u_ip_{33} - p_{13} & u_ip_{34} - p_{14} \\ v'_ip'_{31} - p'_{21} & v'_ip'_{32} - p'_{22} & v'_ip'_{33} - p'_{23} & v'_ip'_{34} - p'_{24} \\ u'_ip'_{31} - p'_{11} & u'_ip'_{32} - p'_{12} & u'_ip'_{33} - p'_{13} & u'_Ip'_{34} - p'_{14} \end{bmatrix}\tilde{\mathbf{X}}_i = 0$$
17. $$A\tilde{\mathbf{X}}_i = 0$$
18. Combining these equations we get an over determined homogeneous system of linear equations that we can solve with SVD.
19. Since we have two views: $\begin{bmatrix} v_i\mathbf{p}_3^T - \mathbf{p}_2^T \\ u_i\mathbf{p}_3^T - \mathbf{p}_1^T \\ {v'}_i\mathbf{p'}_3^T - \mathbf{p'}_2^T \\ {u'}_i\mathbf{p'}_3^T - \mathbf{p'}_1^T \\ \vdots \end{bmatrix}\tilde{\mathbf{X}}_i = 0$
20. $$\begin{bmatrix} v_ip_{31} - p_{21} & v_ip_{32} - p_{22} & v_ip_{33} - p_{23} & v_ip_{34} - p_{24} \\ u_ip_{31} - p_{11} & u_ip_{32} - p_{12} & u_ip_{33} - p_{13} & u_ip_{34} - p_{14} \\ v'_ip'_{31} - p'_{21} & v'_ip'_{32} - p'_{22} & v'_ip'_{33} - p'_{23} & v'_ip'_{34} - p'_{24} \\ u'_ip'_{31} - p'_{11} & u'_ip'_{32} - p'_{12} & u'_ip'_{33} - p'_{13} & u'_Ip'_{34} - p'_{14} \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}\tilde{\mathbf{X}}_i = 0$$
21. The minimized algebraic error is not geometrically meaningful, but the method extends naturally to the case when $\mathbf{X}_i$ is observed in more than two images.

5. method two [11]:
    1. Given $\mathbf{P}$, $\mathbf{P}'$, $(x_i, y_i)$, $(x'_i, y'_i)$, for $i = 1, 2, \ldots, n$.
    2. Solve $(X_i, Y_i, Z_i)$ for $i = 1, 2, \ldots, n$ such that
    3. $\tilde{\mathbf{m}}_i = \mathbf{P}\tilde{\mathbf{M}}_i$ and $\tilde{\mathbf{m}'}_i = \mathbf{P}'\tilde{\mathbf{M}}_i$
    4. the first equation can be written as
    5. $$\begin{bmatrix} \lambda_i x_i \\ \lambda_i y_i \\ \lambda_i \end{bmatrix} = \begin{bmatrix} p_{11} & p_{12} & p_{13} & p_{14} \\ p_{21} & p_{22} & p_{23} & p_{24} \\ p_{31} & p_{32} & p_{33} & p_{34} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$
    6. $$\begin{aligned} (p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34})x_i &= p_{11}X_i + p_{12}Y_i + p_{13}Z_i + p_{14} \\ (p_{31}X_i + p_{32}Y_i + p_{33}Z_i + p_{34})y_i &= p_{21}X_i + p_{22}Y_i + p_{23}Z_i + p_{24} \end{aligned}$$
    7.
6. Of the 4 combinations of rotation and translation [10]:
    1. one produces the correct result.
    2. one produces a reflected version may yield the same number of positive $Z$ points as correct result.
    3. two produce incorrect results.

| [12] | structure (scene geometry) | Motion (camera geometry) | Measurements |
|---|---|---|---|
| Camera Calibration (aka. pose estimation) | known | estimate | 3D to 2D correspondences |
| Triangulation | estimate | known | 2D to 2D correspondences |
| Reconstruction (including epipolar) | estimate | estimate | 2D to 2D correspondences |

References:

1. 3D vision NUS, Prof. Lee Gim Hee, Youtube.
2. Lecture 5, Epipolar Geometry, Introduction to Computer Vision (2025-2), Prof Gyeonsik Moon, Korea University.
3. Computer Vision, Projective Geometry in 2D, Lars Schmidt-Thieme, University of Hidelsheim.
4. camera notation tutorial by Paul Furgale
5. https://www.cvl.isy.liu.se/education/undergraduate/tsbb15/reading-material/RANSAC.pdf
6. Ridham Bansal, The eight point algorithm: Computing the Fundamental Matrix, CS5330 Creative Assessment, Youtube.
7. Visual Odometry Tutorial, AirLab Summer School Series 2020, Yafei Hu.
8. CSIE 5429 3D-DLCV, National Taiwan University, Spring 2021.
9. Lecture 5, Depth Estimation, Introduction to Computer Vision (2025-2), Prof Gyeonsik Moon, Korea University.
10. https://www.comp.nus.edu.sg/~cs4243/lecture/multiview.pdf
11. Two-View Geometry, DVLab, National Sun Yat-sen University.
12. Introduction to computer vision, Atlas Wang, Fall 2022, Utexas.
13. Anatomical-Marker-Driven 3D Markerless Human Motion Capture, Prayook Jatesiktat, et al.